

LLMsVerifier Bridge — Usage Guide

Revision: 1 **Last modified:** 2026-06-15T19:40:00Z **Authority:** Operator mandate 2026-06-15 (LLMsVerifier-only, no local llama.cpp) **Scope:** How every HelixTranslate component, and a Claude Code agent session, obtains and calls the LLMsVerifier-obtained **strongest verified model** through the single pkg/bridge facade — out of the box, with no manual tuning. **Design reference:** docs/design/LLMSVERIFIER_BRIDGE.md (architecture + removal map).

1. What the bridge is

pkg/bridge is the single facade between the HelixTranslate System and the LLMsVerifier-obtained **strongest** verified models. It wraps the EXISTING internal/verifier pipeline + selection.Engine + pkg/translator/llm.VerifiedFactory (no reimplementation of discovery / scoring / selection — §11.4.74) behind four entry points:

Method	Purpose	Used by
BestTranslator(ctx, task)	Strongest verified model as a translator.Translator plus a deterministic score-descending fallback chain.	Every translating component.
Invoke(ctx, system, prompt)	The raw chat capability of the strongest model.	The Claude Code agent (direct model access).
BestModel(ctx, task)	Metadata for the single strongest verified model (no API key).	Agent / diagnostics.
ListVerified(ctx)	All verified models ranked strongest-first (no API keys).	Agent / diagnostics.

“Strongest” = the highest-scoring verified model (FallbackOrder == 1), score-descending, with the rest of the verified set forming the fallback chain (top-1 + fallback — the locked operator decision D4).

2. How a COMPONENT calls it

```
import (
    "digital.vasic.translator/internal/verifier/selection"
```

```

    "digital.vasic.translator/pkg/bridge"
)

b, err := bridge.Open(ctx, bridge.Options{}) // zero value = 00TB
    in-process mode
if err != nil {
    return err // honest hard error if no provider keys are
        present (see §5)
}

tr, fallbacks, err := b.BestTranslator(ctx,
    selection.TaskRequirements{})
if err != nil {
    return err
}
// tr is a translator.Translator backed by the strongest verified
    model.
// fallbacks is a score-descending chain of model IDs to try if tr
    is unreachable.
out, err := tr.Translate(ctx, sourceText, prompt)

```

A component never names a provider, a model, a base URL, or an API key: the bridge selects the strongest verified model and materializes it (provider → base_url → key) automatically. There is **no local llama.cpp / Ollama fallback** — local inference is forbidden by the mandate; if no verified model can be provisioned the bridge returns an honest error.

3. How the AGENT (Claude Code) calls it

Two transports — both out of the box (locked decision D3 = CLI **and** MCP):

3.1 CLI subcommands (cmd/model-bridge, built to ./build/model-bridge)

```

# Build once:
go build -o build/model-bridge ./cmd/model-bridge/

# Strongest model + fallback chain (human-readable; add -json for
    JSON):
./build/model-bridge best-model

# All verified models, JSON, strongest-first:
./build/model-bridge list

# Send a raw prompt to the strongest model (prompt via flag or
    stdin):
./build/model-bridge invoke -prompt "Translate 'good morning' to
    Spanish."
echo "Summarize this in one line: ..." | ./build/model-bridge
    invoke

```

Common bootstrap flags (all subcommands): `-db PATH` (SQLite store, default `./data/verified_models.db`), `-min-score F`, `-max N` (models per provider to verify on a fresh pass; bounds wall-clock + tokens — §11.4.82), `-verify-timeout`, `-force-verify`, `-api-url / -api-key` (HTTP service, see §4).

API-key VALUES are never printed (§11.4.10) — only provider IDs, model IDs, scores, and real model completions.

3.2 MCP stdio server (native agent tools)

`model-bridge mcp` runs a self-contained MCP (Model Context Protocol) stdio server exposing three tools the agent calls natively:

Tool	Arguments	Returns
<code>bridge_invoke</code>	<code>{prompt, system?}</code>	The strongest model’s real completion.
<code>bridge_best_model</code>	<code>{}</code>	Strongest model metadata + fallback chain.
<code>bridge_list</code>	<code>{}</code>	All verified models, strongest-first.

It is wired into the repo-root `.mcp.json`:

```
{
  "mcpServers": {
    "model-bridge": { "command": "./build/model-bridge", "args":
      ["mcp"], "env": {} }
  }
}
```

Build the binary (`go build -o build/model-bridge ./cmd/model-bridge/`), ensure at least one provider key is exported (§5), and restart the Claude Code session so it loads `.mcp.json`. The three `bridge_*` tools then appear as native tools. The MCP server completes the `initialize / tools/list` handshake immediately and provisions the bridge **lazily** on the first tool call that needs a model.

4. Out-of-the-box bootstrap (no manual tuning)

`bridge.Open` resolution order (§11.4.6 — no guessing):

1. **LLMSVERIFIER_API_URL set** → HTTP mode: talk to a running `LLMsVerifier` service (`-api-url / LLMSVERIFIER_API_URL`, Bearer via `-api-key`). The bridge Pings it and fails honestly if unreachable.
2. **Otherwise (canonical OOTB)** → in-process mode: `ProvidersFromGetenv()` discovers every provider whose `*_API_KEY` is set → loads any already-verified models from the SQLite store at `-db` → if none are selectable, runs ONE bounded `RunVerification` pass against the **real** provider APIs (8-step pipeline: reachability / auth / model-existence / response-format / latency /

capabilities / rate-limits / error-handling) → persists passing models → selects the strongest. **No running service is required.**

3. **No *_API_KEY present** → honest hard error naming the env vars to set. There is no local-model fallback (forbidden by the mandate).

The SQLite store holds verified-model **metadata only** — never API keys.

5. Provider keys (§11.4.10 — required everywhere, R2)

Export at least one provider key before using the bridge (any subset of the 16 OpenAI-compatible providers in `internal/verifier/providers_env.go`):

```
export DEEPSEEK_API_KEY=...      # or OPENAI_API_KEY /  
      GROQ_API_KEY / GEMINI_API_KEY / ...
```

Keys are read from the environment **in memory only** and are never logged, printed, or persisted. Per the operator mandate (R2) keys are required on every path — there is no offline / keyless mode, and tests that exercise the real in-process pipeline **fail** (not skip) when keys are genuinely required.

`.env` (git-ignored) is the recommended local store; do not commit keys.

6. Zero tuning, full decoupling

- The provider set is the single source of truth in `internal/verifier`'s `envProviderSpecs`; adding a provider there makes it resolvable with no extra bridge wiring.
 - A verified model carries only a `ProviderID` (a named id in-process, or a numeric id from the HTTP server path). The bridge's `ProviderResolver` materializes (`factoryProvider`, `base_url`, `apiKey`) for both forms — closing the load-bearing “verified model → translating client” gap (design §3.3).
 - Every OpenAI-compatible provider is served by the generic OpenAI-compatible client (which honours `BaseURL`), so providers the legacy factory switch does not natively case (`openrouter` / `together` / `fireworks` / `nvidia` / ...) still work.
-

7. Tests

- `internal/verifier/provider_resolver_test.go` — named-id + numeric-id (HTTP path) + every-provider + missing-key + out-of-range resolution; the numeric-id case is a §11.4.115 RED-then-GREEN regression for the load-bearing bug.
- `pkg/bridge/bridge_test.go` — strongest-first ranking, best-model top-score, score-descending fallback chain, numeric-`ProviderID` end-to-end through the bridge, empty-registry honest error, **no-keys honest hard error**.

- `cmd/model-bridge/mcp_test.go` — MCP `initialize / tools/list / tools/call` routing, an **unforgeable-challenge** (§11.4.78) proving `bridge_invoke` carries the real model round-trip (a per-call nonce), honest tool-error surfacing, and a full byte-stream serve end-to-end.

All three suites are mutation-verified (§1.1): breaking the numeric-id branch, the ranking sort, or the invoke round-trip each makes a test FAIL.

Sources verified

- Model Context Protocol — stdio transport + JSON-RPC 2.0 message shapes (`initialize`, `tools/list`, `tools/call`): <https://modelcontextprotocol.io/docs> (verified 2026-06-15). The MCP server is a self-contained implementation of this wire protocol — no external MCP-framework dependency was added (§11.4.74).